

Primitives and Objects

The seven primitives, every object kind you will meet, and the four rules that split them

October 2026

The split. Every value in TypeScript is either a **primitive** or an **object**. Primitives are atoms: a single value with no parts to reach into. Objects are containers: they have keys, elements or internal state that can be edited. Which side a value is on decides whether it can be mutated, whether two labels can alias it, how `===` compares it, and whether it has its own keys.

The seven primitives

Primitive	Literal	typeof	Notes
string	"abc", 'abc', `a \${x}`	"string"	immutable; every method returns a new string
number	42, 3.5, -0, NaN, Infinity	"number"	one type for integers and floats; NaN is a number
boolean	true, false	"boolean"	
undefined	undefined	"undefined"	never assigned; missing key; no return
null	null	"object"	deliberately empty; the typeof result is a historical wart
bigint	9007199254740993n	"bigint"	integers beyond 2^{53} ; cannot mix with number in arithmetic
symbol	Symbol("id")	"symbol"	unique keys; rare in application code

Python equivalents: `str`, `int/float`, `bool`, `None` (for both `null` and `undefined`), `int` (for `bigint`); no `symbol`.

Four rules that hold for every primitive

- Immutable.** No operation edits a primitive in place. `s.toUpperCase()` returns a new string; `s[0] = "x"` is silently ignored; `n++` reassigns `n` to a new number.
- Compared by value.** `"a" === "a"` is `true`; `5 === 5` is `true`. The one exception is `NaN === NaN`, which is `false`; use `Number.isNaN`.
- Copied on assignment.** `let y = x` gives `y` its own value; nothing done to `y` affects `x`. Passing a primitive to a function cannot change the caller's variable.
- No own keys.** `n.foo = 1` is silently ignored. `"abc".length` and `"abc".toUpperCase()` work because the engine wraps the primitive in a temporary object for the call, then discards it.

The objects

Everything that is not one of the seven above. All objects share the opposite four rules: mutable (unless frozen), compared by identity, aliased on assignment, and able to carry keys.

Core containers

Object	Literal / constructor	typeof	Python	Notes
plain object	{ a: 1 }, new Object()	"object"	dict	string keys; inherits from <code>Object.prototype</code>
array	[1, 2], new Array(3), <code>Array.from(x)</code>	"object"	list	<code>Array.isArray(x)</code> is the only reliable check
Map	<code>new Map([["k", 1]])</code>	"object"	dict	any key type; no prototype keys; insertion order
Set	<code>new Set([1, 2])</code>	"object"	set	unique values
WeakMap, WeakSet	<code>new WeakMap()</code>	"object"		keys are objects, held weakly; not iterable
<code>Object.create(null)</code>		"object"		plain object with no prototype; safe for user keys

Functions (objects that can be called)

Object	Literal	typeof	Notes
function	<code>function f() {}</code> , <code>() => {}</code> , <code>async () => {}</code>	"function"	the only object with <code>typeof</code> other than "object"; can have properties
class	<code>class A {}</code>	"function"	a constructor function; <code>new A()</code> makes an instance
generator	<code>function* g() {}</code>	"function"	

Built-in wrappers and values

Object	Constructor	Notes
Date	<code>new Date()</code> , <code>new Date(0)</code>	mutable (<code>setDate</code>); compare with <code>.getTime()</code> ; <code>Date.now()</code> is a primitive number
RegExp	<code>/ab+/g</code> , <code>new RegExp("ab+")</code>	with the <code>g</code> flag it carries <code>lastIndex</code> state between calls
Error and subclasses	<code>new Error("m")</code> , <code>new TypeError()</code>	<code>.message</code> , <code>.stack</code> , <code>.cause</code> ; <code>catch (e)</code> gives <code>unknown</code>
Promise	<code>new Promise(...)</code> , any <code>async</code> call	a ticket; unwrap with <code>await</code>
class instance	<code>new User()</code>	has the class's methods via its prototype
String, Number, Boolean objects	<code>new String("a")</code>	wrapper OBJECTS, never used deliberately; <code>new String("a") === "a"</code> is <code>false</code>

Binary and typed data

Object	Constructor	Notes
ArrayBuffer	<code>new ArrayBuffer(8)</code>	raw bytes
typed arrays	<code>new Uint8Array(4)</code> , <code>Float64Array</code>	fixed-type numeric arrays
Buffer (Node)	<code>Buffer.from("abc")</code>	Node's byte array
Blob, File (browser)		

Iteration and async helpers

Object	Notes
iterator / generator object	what <code>function*</code> returns; <code>.next()</code>
AbortController, AbortSignal	cancel a <code>fetch</code>
URL, URLSearchParams	parsed URL; mutable
Headers, Request, Response	the <code>fetch</code> family
FormData	

Framework objects you will meet in handlers

Object	Notes
Express <code>req</code>	<code>req.params</code> , <code>req.query</code> , <code>req.body</code> , <code>req.headers</code> are all objects; their values are strings, arrays or <code>any</code>
Express <code>res</code>	<code>res.status()</code> , <code>res.json()</code> , <code>res.send()</code> are methods; calling them does not end the function
database client / pool	every query method returns a Promise
EventEmitter	<code>.on</code> , <code>.off</code> , <code>.emit</code> ; listeners never removed are leaks

Telling them apart at runtime

Check	Primitive	Object
<code>typeof x</code>	"string", "number", "boolean", "undefined", "bigint", "symbol"	"object" or "function" (and <code>null</code> wrongly says "object")
<code>x === null</code>	catches <code>null</code>	
<code>Array.isArray(x)</code>		<code>true</code> for arrays only
<code>x instanceof Date / Map / Error / MyClass</code>		<code>true</code> when built by that constructor
<code>Object(x) === x</code>	<code>false</code>	<code>true</code>

A reliable “is this a primitive” test: `x === null || (typeof x !== "object" && typeof x !== "function")`.

The four rules side by side

Rule	Primitive	Object
Can be edited in place?	never	yes (unless <code>Object.freeze</code> , and then only one level)
<code>===</code> compares	the value	the address (identity)

Rule	Primitive	Object
<code>let y = x</code> gives	an independent value	a second label on the same thing
Own keys?	none; <code>x.foo = 1</code> ignored	yes; <code>x.foo = 1</code> adds a key
Passed to a function	the function cannot change the caller's variable	the function can mutate the caller's object
Review question	was the method's result stored?	who else is holding this? which methods mutate?

Where the split shows up in the sheets

Topic	Primitive side	Object side
Mutability (its own sheet)	every method returns new; call-then-assign	mutate vs return-new tables
Memory model	observably copied	arrows shared
Equality (Behaviors 8)	<code>===</code> on values	<code>===</code> on identity; compare contents by hand
Truthiness (Obstacles note B)	<code>0</code> , <code>" "</code> , <code>NaN</code> falsy	every object truthy, including <code>[]</code> and <code>{}</code>
Map keys (Obstacles note J)	<code>1</code> and <code>"1"</code> are different keys	an object key is matched by identity only
<code>undefined</code> / <code>null</code>	both are primitives	following an arrow to them is the crash

One sentence to keep.

Seven primitives (string, number, boolean, undefined, null, bigint, symbol) are immutable, compared by value, copied on assignment and have no keys; everything else is an object and is the opposite on all four.